



Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών
Πανεπιστήμιο Θεσσαλίας

ECE333 - Εργαστήριο Ψηφιακών Συστημάτων

Χειμερινό Εξάμηνο — Ακαδημαϊκό Έτος 2025-2026

Εργαστηριακή Εργασία 4^η

Υλοποίηση παιχνιδιού Pong

9/12/2025 έως 16/01/2026

Δ. Γαρυφάλλου

Περιεχόμενα

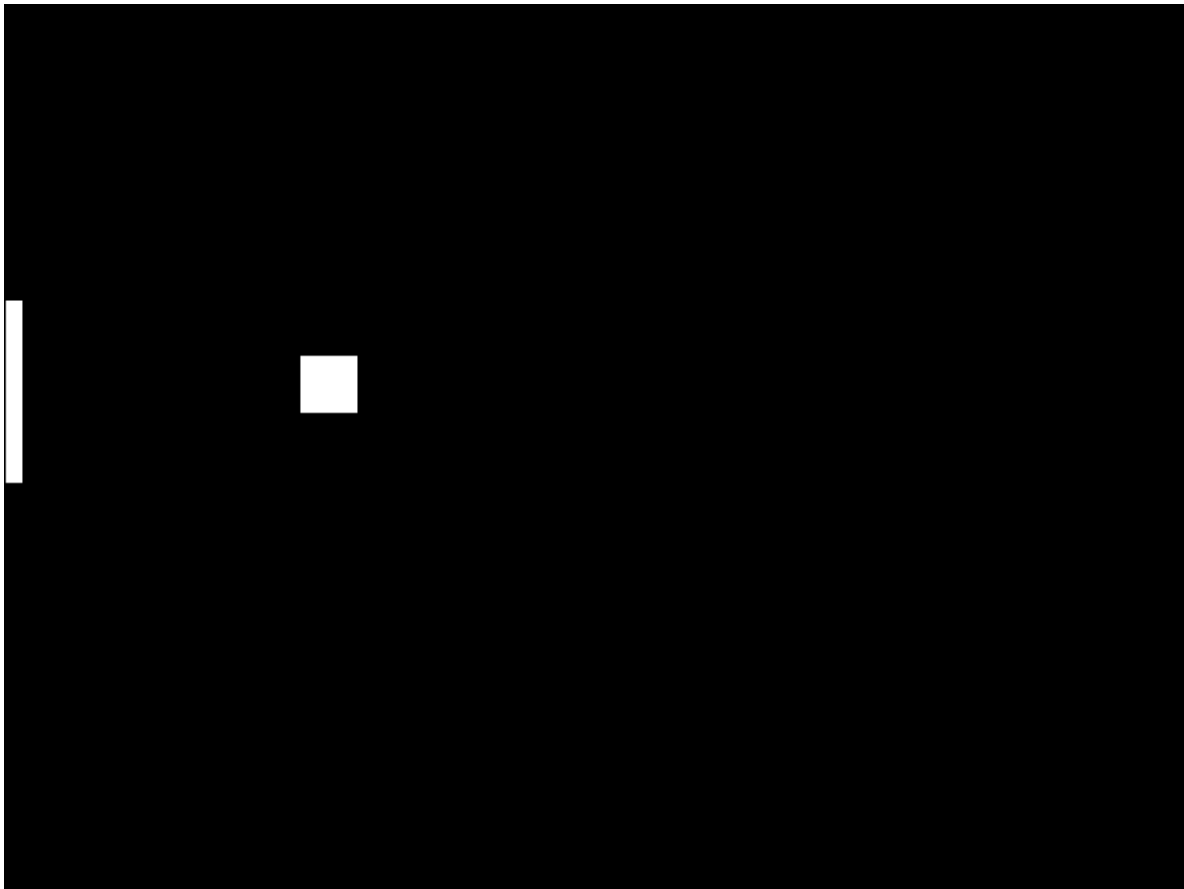
1	Στόχος της 4 ^{ης} Εργασίας	2
2	Το παιχνίδι Pong	2
2.1	Ο μηχανισμός των Sprites	3
2.2	Ανανέωση των Frames	4
2.3	Κίνηση Ρακέτας (Paddle)	4
2.4	Κίνηση Δίσκου (Puck)	4
2.5	Ταχύτητα κίνησης Ρακέτας και Δίσκου	5
2.6	Τέλος Παιχνιδιού (Game Over)	5
2.7	Παραμετροποίηση και Προσομοίωση κυκλώματος	5
3	Μέρος Α - Οδήγηση και επαλήθευση σήματος new_frame	6
4	Μέρος Β - Απεικόνιση Block Sprites	7
5	Μέρος Γ - Δομή μονάδας Pong & Κίνηση Ρακέτας (Paddle)	8
6	Μέρος Δ - Κίνηση δίσκου (Puck) και ολοκλήρωση του παιχνιδιού Pong	9
7	Επέκταση του παιχνιδιού Pong (Προαιρετικό)	10
8	Προθεσμία Παράδοσης, Υποβολή της Εργασίας και Αναφορά	11
9	Ερωτήσεις και Απορίες	11

1 Στόχος της 4^{ης} Εργασίας

Ο στόχος της 4^{ης} εργαστηριακής εργασίας είναι η υλοποίηση του arcade παιχνιδιού **Pong**, σε ανάλυση 640 × 480p και συχνότητα ανανέωσης 60Hz, χρησιμοποιώντας τον ελεγκτή HDMI που σχεδιάστηκε στα πλαίσια της 3^{ης} εργασίας.

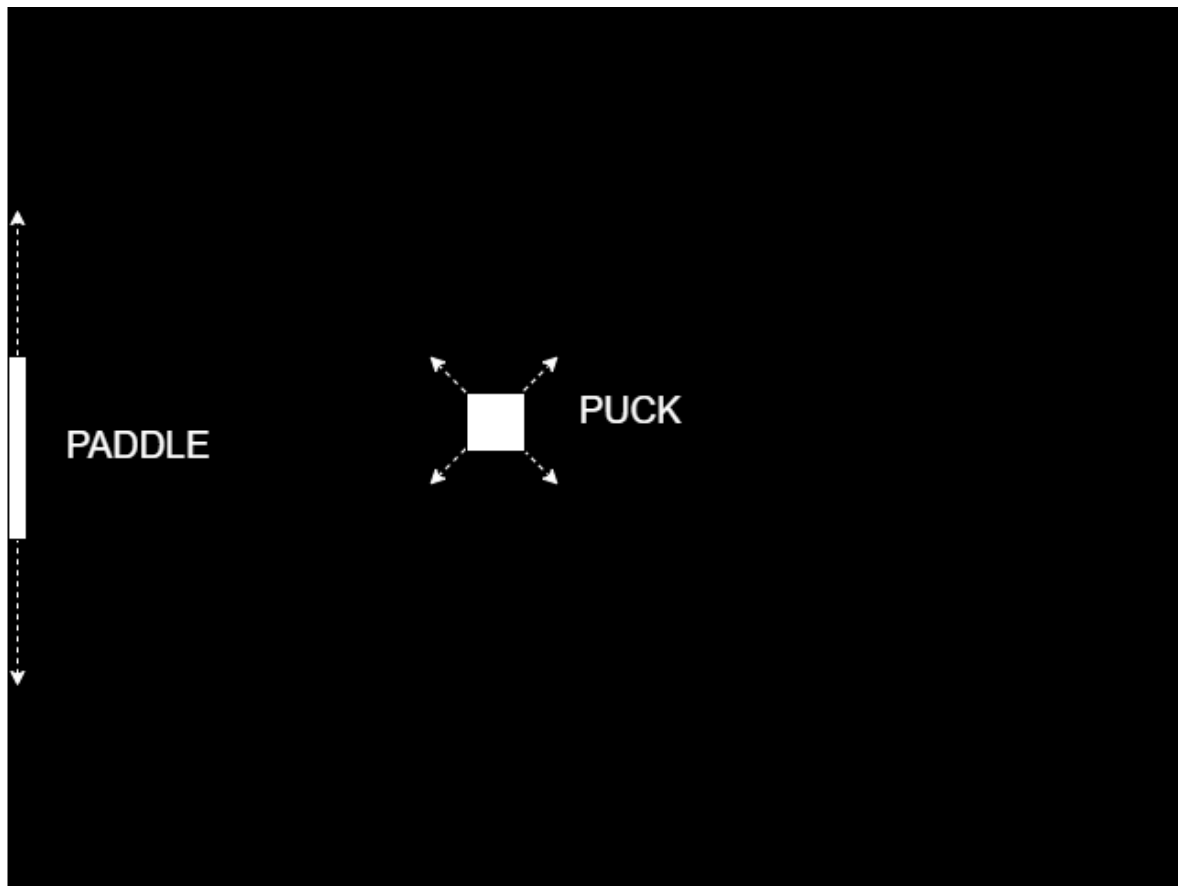
2 Το παιχνίδι Pong

Το Pong είναι ένα από τα πρώτα ηλεκτρονικά παιχνίδια που κατασκευάστηκαν, και η λογική του είναι απλή. Αν και η αρχική υλοποίηση του παιχνιδιού αφορούσε δύο παίκτες που παίζουν ως αντίπαλοι, στο πλαίσιο της εργασίας θα κατασκευαστεί μία εκδοχή του που αφορά έναν παίκτη. Στο Σχήμα 1 φαίνεται ένα στιγμιότυπο από το παιχνίδι Pong που θα κατασκευαστεί.



Σχήμα 1: Ένα στιγμιότυπο του παιχνιδιού Pong , με διαστάσεις υπό κλίμακα

Αποτελείται από δύο αντικείμενα, τη ρακέτα (Paddle) και τον δίσκο (Puck) τα οποία εμφανίζονται με τη μορφή **block sprites** στην οθόνη και αλληλεπιδρούν μεταξύ τους. Ο παίκτης καλείται να μετακινεί τη ρακέτα σε κάθετο προσανατολισμό στην αριστερή μεριά της οθόνης, προσπαθώντας να αποτρέψει το δίσκο από το να έρθει σε επαφή με την αριστερή πλευρά. Ο δίσκος κινείται διαγώνια εντός όλης της οθόνης, προς όλες τις κατευθύνσεις, ακολουθώντας σταθερή πορεία μέχρι να έρθει σε επαφή είτε με κάποιον από τους τρεις τοίχους που επιτρέπεται να αγγίζει (πάνω, κάτω και δεξιά), είτε με τη ρακέτα. Εάν γίνει αυτό, ο δίσκος εξοστραχίζεται και κινείται προς την αντίθετη κατεύθυνση. Στο Σχήμα 2 φαίνεται ο τρόπος με τον οποίο κινούνται οι δύο οντότητες του παιχνιδιού.



Σχήμα 2: Το εύρος κινήσεων της Ρακέτας και του Δίσκου

Στην περίπτωση που ο παίκτης δεν καταφέρει να αποτρέψει το δίσκο από το να αγγίξει τον αριστερό τοίχο, το παιχνίδι τελειώνει. Πρόκειται, δηλαδή, για ένα παιχνίδι που δοκιμάζει τα αντανακλαστικά του παίκτη και την ικανότητά του να υπολογίσει γρήγορα την τροχιά του δίσκου, ώστε να κινήσει κατάλληλα τη ρακέτα.

2.1 Ο μηχανισμός των Sprites

Τα Sprites αποτελούν βασική τεχνική γραφικής απεικόνισης σε ψηφιακά συστήματα, όπου ένα κινούμενο ή στατικό αντικείμενο δεν αποθηκεύεται ως τμήμα μιας συνολικής εικόνας αλλά σχεδιάζεται δυναμικά, με βάση τις τρέχουσες συντεταγμένες του κατά την παραγωγή κάθε pixel. Σε αντίθεση με την εμφάνιση εικόνων από μνήμη, όπου όλη η πληροφορία βρίσκεται αποθηκευμένη και προσπελαύνεται σειριακά, υπολογίζονται σε πραγματικό χρόνο μέσω λογικών συγκρίσεων: για κάθε pixel που σαρώνει ο Video Controller, το σύστημα αποφασίζει αν αυτό το pixel πρέπει να δείξει το χρώμα του sprite ή το χρώμα του φόντου. Έτσι, η μετακίνηση ενός sprite γίνεται απλώς αλλάζοντας τις συντεταγμένες του, χωρίς να χρειάζεται ανανέωση ή μεταγραφή μιας ολόκληρης εικόνας στη μνήμη. Αυτή η προσέγγιση είναι ιδιαίτερα αποτελεσματική σε υλοποιήσεις FPGA, όπου οι διαθέσιμοι πόροι για μνήμες είναι περιορισμένοι.

Τα sprites που θα πρέπει να κατασκευαστούν στο συγκεκριμένο παιχνίδι είναι τα εξής δύο:

- Της ρακέτας, το οποίο θα έχει ορθογώνιο σχήμα

- Του δίσκου το οποίο θα έχει τετράγωνο σχήμα

Η χρήση τετράπλευρων σχημάτων με σταθερό μήκος και πλάτος διευκολύνει τον έλεγχο που πραγματοποιείται για κάθε pixel. Και τα δύο `sprites` προτείνεται να έχουν άσπρο χρώμα, για να διακρίνονται από το μαύρο φόντο. Επισημαίνεται ότι τα δύο αντικείμενα θα καταλαμβάνουν συνεχώς διαφορετικά και ανεξάρτητα μεταξύ τους τμήματα της οθόνης, καθώς λόγω του μηχανισμού εξοστρακισμού ποτέ δεν θα επικαλύπτει το ένα το άλλο.

2.2 Ανανέωση των Frames

Παρότι, ως ψηφιακό κύκλωμα, το παιχνίδι Pong βασίζεται στη χρήση ρολογιού (`pixel clock`), η λογική του παιχνιδιού δεν πρέπει να ενημερώνεται με βάση το ρολόι. Κάτι τέτοιο θα οδηγούσε σε υπερβολική και μη ρεαλιστική κίνηση. Αντίθετα, η ανανέωση γίνεται μία φορά ανά καρέ (`frame`), συνήθως όταν οι μετρητές βρίσκονται στο μη ορατό σημείο του `frame`. Με αυτόν τον τρόπο, η ταχύτητα του παιχνιδιού παραμένει σταθερή και ρεαλιστική.

2.3 Κίνηση Ρακέτας (Paddle)

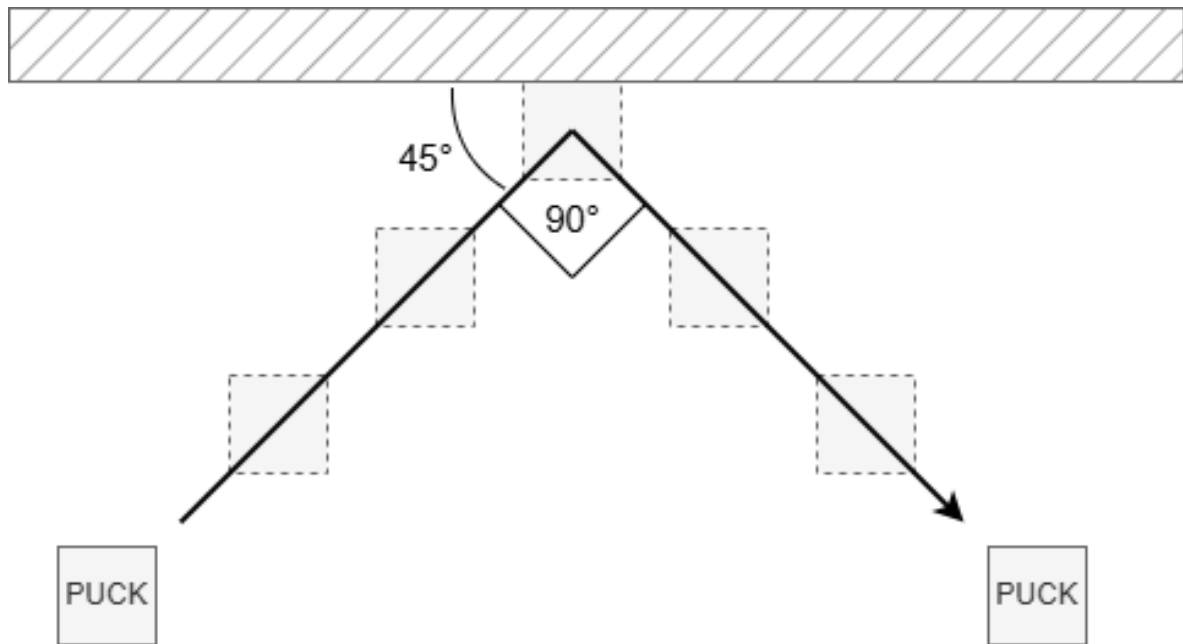
Η κίνηση της ρακέτας πραγματοποιείται μόνο στον κάθετο άξονα, και ελέγχεται εξ' ολοκλήρου από τον χρήστη, με την χρήση κουμπιών που αντιστοιχούν στην κίνηση προς τα πάνω (`up button`) και στην κίνηση προς τα κάτω (`down button`). Η θέση του δίσκου στον οριζόντιο άξονα είναι σταθερή, στο αριστερό άκρο της οθόνης. Όταν ο χρήστης πιέζει το κουμπί `up`, η θέση της ρακέτας θα πρέπει να μετατοπίζεται προς τα πάνω, ενώ όταν πιέζει το κουμπί `down`, η ρακέτα θα κινείται προς τα κάτω. Η κάθετη κίνηση θα διαρκεί για όσο το αντίστοιχο κουμπί είναι ενεργό. Μοναδική εξαίρεση αποτελεί η περίπτωση στην οποία η ρακέτα πρόκειται να βγει εκτός των ορίων της οθόνης με το πάτημα του κουμπιού. Σε αυτή την περίπτωση η θέση της ρακέτας δεν πρέπει να μεταβάλλεται.

2.4 Κίνηση Δίσκου (Puck)

Η κίνηση του δίσκου πραγματοποιείται τόσο στον οριζόντιο όσο και στον κάθετο άξονα. Ωστόσο οι δύο αυτές κινήσεις είναι μεταξύ τους ανεξάρτητες. Όσο το παιχνίδι βρίσκεται σε εξέλιξη, ο δίσκος κινείται συνεχώς και στις δύο κατευθύνσεις με την ίδια ταχύτητα, επομένως πραγματοποιεί διαγώνια κίνηση. Ιδιαίτερα σημαντικός είναι ο χειρισμός των προσκρούσεων στους τοίχους και στη ρακέτα. Όταν ο δίσκος πρόκειται να βγει εκτός ορατού πεδίου, από πάνω, κάτω ή δεξιά, ή όταν έρχεται σε επαφή με τη ρακέτα, θα πρέπει να εξοστραχίζεται και να συνεχίζει τη διαγώνια πορεία του. Αυτό σημαίνει πως

- Εάν η πρόσκρουση γίνεται με τον πάνω ή κάτω τοίχο, η οριζόντια κίνηση παραμένει σταθερή και η κάθετη φορά αντιστρέφεται
- Εάν η πρόσκρουση γίνεται με τον δεξιά τοίχο ή με τη ρακέτα, η κάθετη κίνηση παραμένει σταθερή και η οριζόντια φορά αντιστρέφεται

Στο Σχήμα 3 φαίνεται η γωνία με την οποία ο δίσκος προσκρούει σε μία επιφάνεια, αλλά και η γωνία με την οποία κατευθύνεται αντίθετα. Όπως φαίνεται, η πρόσκρουση γίνεται πάντα με γωνία 45 μοιρών μεταξύ της κατεύθυνσης του δίσκου και της επιφάνειας πρόσκρουσης, και η κίνηση του δίσκου μετά τον εξοστρακισμό γίνεται με γωνία 90 μοιρών σε σχέση με την κίνηση πριν. Το παραπάνω ισχύει γιατί, όπως θα εξηγηθεί παρακάτω, το μέτρο της ταχύτητας στον οριζόντιο και στον κάθετο άξονα είναι πάντα το ίδιο, τουλάχιστον στη συγκεκριμένη υλοποίηση.



Σχήμα 3: Πρόσκρουση του δίσκου στον τοίχο

2.5 Ταχύτητα κίνησης Ρακέτας και Δίσκου

Όταν κάποιο από τα δύο **sprites** μετακινείται, η θέση του (είτε οριζόντια είτε κάθετη) αλλάζει κατά έναν αριθμό από pixel ο οποίος εξαρτάται από την ταχύτητα που επιθυμούμε να έχει. Η πιο απλή υλοποίηση είναι τα **sprites** να κινούνται με ταχύτητα ένα pixel ανά frame, οπότε σε κάθε frame η μεταβλητή που αντιστοιχεί στη θέση του **sprite** αυξάνεται ή μειώνεται κατά ένα pixel. Αύξηση της τιμής μεταφράζεται σε κίνηση προς τα κάτω ή προς τα δεξιά, όσον αφορά τον κάθετο και τον οριζόντιο προσανατολισμό αντίστοιχα, ενώ μείωση της τιμής αντιστοιχεί σε κίνηση προς τα πάνω ή προς τα αριστερά. Εάν η ταχύτητα είναι διαφορετική από ένα pixel ανά frame, οι τιμές των συντεταγμένων αυξάνονται κατά τον εκάστοτε αριθμό των pixels ανά frame, ωστόσο ο έλεγχος για κίνηση των **sprites** εκτός της ορατής εικόνας θα πρέπει να πραγματοποιείται με μεγαλύτερη προσοχή.

2.6 Τέλος Παιχνιδιού (Game Over)

Το παιχνίδι τελειώνει όταν ο δίσκος έρθει σε επαφή με τον αριστερό τοίχο, δηλαδή όταν η ρακέτα δεν καταφέρει να το απωθήσει, όπως φαίνεται στο Σχήμα 4. Όταν το παιχνίδι τελειώνει, τα **sprites** θα διατηρούν μία σταθερή θέση. Ο χρήστης θα πρέπει να έχει τη δυνατότητα, με το πάτημα κάποιου κουμπιού, να ξεκινά καινούριο παιχνίδι από την αρχή.

2.7 Παραμετροποίηση και Προσομοίωση κυκλώματος

Εφόσον το κύκλωμα του παιχνιδιού Pong εξελίσσεται σε επίπεδο frames, η επαλήθευση του κυκλώματος μέσω προσομοίωσης είναι ιδιαίτερα χρονοβόρα. Συνίσταται έντονα να γίνει η χρήση παραμέτρων, τόσο για τις τιμές των περιοχών του HDMI (**active draw**, **back porch**, **front porch** κ.λ.π.), όσο και για το μήκος και το πλάτος των **Sprites**. Με αυτόν τον τρόπο, η προσομοίωση μπορεί να γίνει με αρκετά μικρή και μη ρεαλιστική ανάλυση οθόνης, όπως 20×20 , και τα **stripes** μπορούν να έχουν μήκος και πλάτος λιγότερα από 5 pixel. Αυτή η πρακτική μπορεί να εφαρμοστεί, καθώς η λειτουργία του ελεγκτή HDMI έχει



Σχήμα 4: Στις Περιπτώσεις 2, 3, 4, 5, ο δίσκος θα εξοστραχιστεί από τη ρακέτα. Στις Περιπτώσεις 1 και 6, ο Δίσκος έρχεται σε επαφή με τον αριστερό τοίχο, άρα έχουμε Τέλος Παιχνιδιού (Game Over)

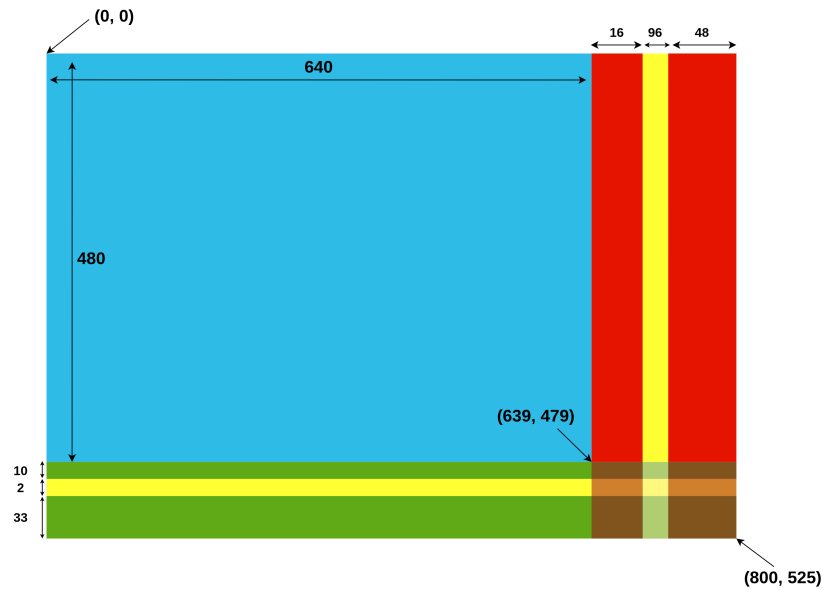
ήδη επαληθευτεί για τις σωστές παραμέτρους, άρα στην προσομοίωση μπορεί να εξεταστεί αποκλειστικά η λογική του παιχνιδιού, η οποία θεωρητικά δεν εξαρτάται από τις διαστάσεις του περιβάλλοντος και των `sprites`. Η χρήση των παραμέτρων ενδείκνυται για διευκόλυνση στη μετάβαση μεταξύ των πραγματικών διαστάσεων και των διαστάσεων προσομοίωσης.

3 Μέρος Α - Οδήγηση και επαλήθευση σήματος `new_frame`

Χρησιμοποιώντας τα σήματα `hcount`, `vcount` του ελεγκτή HDMI, οδηγήστε ένα σήμα `new_frame`, το οποίο θα είναι ενεργό για έναν κύκλο του `pixel clock` ανά frame. Η ενεργοποίηση θα πρέπει να γίνεται όταν το τρέχον `pixel` είναι το τελευταίο της περιοχής `active draw` που διατρέχεται. Επομένως, για ανάλυση $640 \times 480p$, οι τιμές του `hcount` και `vcount` θα πρέπει να είναι 639 και 479 αντίστοιχα, όπως φαίνεται στο Σχήμα 5

Επιπλέον, κάνοντας χρήση του παραπάνω σήματος, επαληθεύστε ότι η συχνότητα του ελεγκτή HDMI είναι πράγματι 60 frames το δευτερόλεπτο. Με τη χρήση ενός μετρητή, προγραμματίστε ένα από τα LED της πλακέτας (όπως το LD0) ώστε να αναβοσβήνει κάθε 60 εμφανίσεις του σήματος `new_frame`, και χρονομετρήστε ότι το χρονικό διάστημα μεταξύ της αλλαγής κατάστασης του LED (από ανοιχτό σε κλειστό και το αντίστροφο) είναι περίπου ίσο με ένα δευτερόλεπτο.

Όταν η κυκλωματική υλοποίηση είναι ορθή, επιδείξτε τον κώδικα Verilog που γράψατε για το κύκλωμα (RTL Design) και το πλαίσιο δοκιμής (Testbench), τα αποτελέσματα της προσομοίωσης, και το κύκλωμα εν λειτουργία σε επιτηρητή του εργαστηρίου.



Σχήμα 5: Υπενθύμιση των χρονικών περιοχών ενός HDMI Frame. Το σήμα `new_frame` πρέπει να ενεργοποιείται στο pixel (639, 479).

4 Μέρος Β - Απεικόνιση Block Sprites

Υλοποιήστε ένα εξ' ολοκλήρου συνδυαστικό κύκλωμα `block_sprite` το οποίο απο-φασίζει, για κάθε pixel που λαμβάνει από τον ελεγκτή HDMI μέσω των σημάτων `hcount` και `vcount`, αν αυτό το pixel βρίσκεται μέσα στα όρια του `stripe` με επάνω-αριστερή γωνία τις συντεταγμένες `sprite_x` και `sprite_y` και διαστάσεις `SPRITE_WIDTH`×`SPRITE_HEIGHT`, οι οποίες θα πρέπει να ορίζονται ως παράμετροι. Αν το τρέχον pixel βρίσκεται εντός του `stripe`, τότε το κύκλωμα παράγει στην έξοδο τα τρία χρωματικά κανάλια RGB που αντιστοιχούν στο χρώμα του `stripe`, το οποίο μπορεί να είναι λευκό (1, 1, 1), ενώ αν βρίσκεται εκτός, δίνει ως έξοδο το χρώμα του φόντου, το οποίο είναι μαύρο (0, 0, 0).

Η προτεινόμενη μορφή του κυκλώματος `block_sprite` φαίνεται στο Σχήμα 6

```
module block_sprite(
    input [10:0] hcount_in,
    input [9:0] vcount_in,
    input [10:0] sprite_x,
    input [9:0] sprite_y,
    output [7:0] red_out,
    output [7:0] green_out,
    output [7:0] blue_out);

    end

    ...

endmodule
```

Σχήμα 6: Μια Μορφή Υλοποίησης του κυκλώματος ελέγχου ενός Block Sprite

Προσαρμόστε κατάλληλα το κύκλωμα του ελεγκτή HDMI και ενώστε σε αυτό το κύκλωμα `block_sprite`. Πλέον θα πρέπει στην οθόνη να μην απεικονίζεται η σταθερή εικόνα της

VRAM, αλλά η εικόνα που προκύπτει από την τρέχουσα θέση των `sprites`. Σε αυτό το βήμα, οι εισόδοι `sprite_x`, `sprite_y` θα περιέχουν μία σταθερή τιμή που θα αντιστοιχεί στην αρχική θέση των `sprites`. Προτείνεται και τα δύο `sprites` να αρχικοποιηθούν περίπου στη μέση του χώρου κίνησής τους.

Το μήκος και πλάτος των `sprites` παρουσιάζεται στον παρακάτω πίνακα:

Παράμετρος	Τιμή
<code>puck_width</code>	32
<code>puck_height</code>	32
<code>paddle_width</code>	10
<code>paddle_height</code>	100

Όταν η κυκλωματική υλοποίηση είναι ορθή, επιδείξτε τον κώδικα Verilog που γράψατε για το κύκλωμα (RTL Design) και το πλαίσιο δοκιμής (Testbench), τα αποτελέσματα της προσομοίωσης, και το κύκλωμα εν λειτουργία σε επιτηρητή του εργαστηρίου.

5 Μέρος Γ - Δομή μονάδας Pong & Κίνηση Ρακέτας (Paddle)

Η λειτουργικότητα του Pong μπορεί να υλοποιηθεί εντός ενός ομώνυμου `module pong`, μία ενδεικτική υλοποίηση του οποίου παρουσιάζεται στο Σχήμα 7. Εσωτερικά θα εμφανίζονται κατάλληλα τα δύο `sprites`, και θα χειρίζεται η κίνηση της ρακέτας, και στο επόμενο βήμα του δίσκου. Υπενθυμίζεται ότι το παρακάτω κύκλωμα θα πρέπει να ενωθεί κατάλληλα με τον υπάρχοντα ελεγκτή HDMI.

```
module pong (
    input clk, // pixel clock //
    input rst,
    input up,
    input down,
    input [3:0] paddle_speed,
    input new_frame,
    input [10:0] hcount,
    input [10:0] vcount,
    output [7:0] red,
    output [7:0] green,
    output [7:0] blue
);

...

endmodule
```

Σχήμα 7: Μια Μορφή Υλοποίησης του κυκλώματος ελέγχου του παιχνιδιού Pong

Σε αυτό το μέρος θα υλοποιήσετε την κάθετη κίνηση της ρακέτας, ενώ ο δίσκος θα παραμένει προς το παρόν ακίνητος. Η κατεύθυνση θα επιλέγεται με δύο από τα κουμπιά της πλακέτας, όπως τα `BTN2` και `BTN3`, τα οποία αντιστοιχούν στις εισόδους `up` και `down`.

Όταν κάποιο από αυτά πιέζεται, η ρακέτα θα πρέπει να κινείται κάθετα προς την αντίστοιχη κατεύθυνση, για όσο μένει πατημένο το κουμπί. Θα πρέπει να μεριμνήσετε για την αποφυγή της μετάβασης της ρακέτας εκτός του ορατού πεδίου της οθόνης.

Η ταχύτητα κίνησης θα εξαρτάται από την είσοδο `paddle_speed`, η οποία θα ελέγχεται με τέσσερα από τα `Switches` της πλακέτας, όπως τα SW12 - SW15. Σε αυτά θα αντιστοιχεί μία τιμή από το 0 ως το 15, η οποία θα εκφράζει την ταχύτητα της ρακέτας, σε μονάδα pixels ανά frame. Εάν η ταχύτητα είναι 0 σημαίνει πως η ρακέτα είναι ακίνητη. Η ταχύτητα μπορεί να μεταβάλλεται όσο η ρακέτα είναι σε κίνηση. Δώστε ιδιαίτερη προσοχή στην παραμονή της ρακέτας εντός της οθόνης όταν η ταχύτητα είναι μεγαλύτερη από 1.

Επιπλέον, ιδιαίτερη προσοχή πρέπει να δοθεί ώστε η κίνηση να γίνεται μόνο στον κύκλο όπου το σήμα `new_frame` είναι ενεργό. Σε κάθε άλλο κύκλο του `pixel clock`, όλα τα στοιχεία της οθόνης θα πρέπει να παραμένουν σταθερά.

Όταν η κυκλωματική υλοποίηση είναι ορθή, επιδείξτε τον κώδικα Verilog που γράψατε για το κύκλωμα (RTL Design) και το πλαίσιο δοκιμής (Testbench), τα αποτελέσματα της προσομοίωσης, και το κύκλωμα εν λειτουργία σε επιτηρητή του εργαστηρίου.

6 Μέρος Δ - Κίνηση δίσκου (Puck) και ολοκλήρωση του παιχνιδιού Pong

Υλοποιήστε τη συνεχή διαγώνια κίνηση του δίσκου, η οποία όπως αναφέρθηκε παραπάνω αποτελείται από μία οριζόντια και μία κάθετη συνιστώσα, οι οποίες είναι ανεξάρτητες μεταξύ τους.

Υλοποιήστε επίσης το μηχανισμό εξοστραχισμού του, δίνοντας προσοχή στη συνθήκη κατά την οποία υπάρχει εξοστραχισμός από τη ρακέτα. Το παιχνίδι τελειώνει μόνο όταν στην αριστερή πλευρά δεν υπάρχει καμία επαφή της ρακέτας με το δίσκο. Αν υπάρχει έστω και η ελάχιστη επαφή (π.χ ενός pixel) τότε θα πραγματοποιείται εξοστραχισμός, με τον τρόπο που εξηγήθηκε παραπάνω.

Προσθέστε στο module pong τις δύο επιπλέον εισόδους που φαίνονται στο Σχήμα 8:

```
input [3:0] puck_speed
input new_game
```

Σχήμα 8: Επιπρόσθετα σήματα εισόδου για το κύκλωμα Pong

Το σήμα `puck_speed` είναι αντίστοιχο του `paddle_speed`, δηλαδή ελέγχει την ταχύτητα του δίσκου, και μπορεί να ελέγχεται με τη χρήση 4 `Switches`, όπως τα SW8 - SW11. Και σε αυτή την περίπτωση, η τιμή της ταχύτητας θα πρέπει να μπορεί να μεταβάλλεται όσο το παιχνίδι είναι σε εξέλιξη.

Όταν ένα παιχνίδι τελειώνει (`game over`) η εικόνα θα πρέπει να είναι σταθερή. Καινούριο παιχνίδι θα ξεκινάει με την ενεργοποίηση της εισόδου `new_game`, η οποία θα γίνεται με το πάτημα κάποιου κουμπιού όπως το BTN1. Όταν ξεκινάει ένα παιχνίδι, η αρχική κίνηση του δίσκου θα πρέπει να είναι διαγώνια, όπως φαίνεται στο Σχήμα 2, χωρίς να έχει σημασία προς τα πού γίνεται αυτή. Μπορείτε να υλοποιήσετε είτε συγκεκριμένη κατεύθυνση αρχικής κίνησης, είτε "ψευδοτυχαία" χρησιμοποιώντας τα σήματα `hcount[0]` και `hcount[1]`, τα οποία έχουν κάποια μη γνωστή τιμή τη στιγμή που ξεκινάει το νέο παιχνίδι.

Σε αυτό το σημείο θα πρέπει το παιχνίδι Pong να είναι έτοιμο. Επιβεβαιώστε, δοκιμάζοντας διαφορετικές ταχύτητες και αλλάζοντάς τες κατά τη διάρκεια του παιχνιδιού, ότι όλες οι λειτουργίες δουλεύουν σωστά, δηλαδή ότι κανένα από τα Sprites δεν βγαίνει εκτός του ορατού πεδίου, ότι ο εξοστρακισμός γίνεται προς τη σωστή κατεύθυνση, και ότι η ρακέτα αποκρούει κατάλληλα το δίσκο, καθώς και ότι η αρχικοποίηση του παιχνιδιού στην περίπτωση έναρξης από την αρχή (New Game) γίνεται σωστά.

Όταν η κυκλωματική υλοποίηση είναι ορθή, επιδείξτε τον κώδικα Verilog που γράψατε για το κύκλωμα (RTL Design) και το πλαίσιο δοκιμής (Testbench), τα αποτελέσματα της προσομοίωσης, και το κύκλωμα εν λειτουργία σε επιτηρητή του εργαστηρίου.

7 Επέκταση του παιχνιδιού Pong (Προαιρετικό)

Προαιρετικά, μπορείτε να προσθέσετε επιπλέον λειτουργίες στο παιχνίδι που κατασκευάσατε, είτε αξιοποιώντας κυκλώματα ή υποκυκλώματα που έχουν δημιουργηθεί σε προηγούμενες εργασίες, είτε επεκτείνοντας με καινούρια λογική. Παρακάτω παρουσιάζονται κάποιες ενδεικτικές ιδέες, αλλά μπορείτε να υλοποιήσετε και κάποια δική σας:

- Αξιοποιώντας το κύκλωμα του 7-Segment Display, μπορείτε να κρατάτε σκορ και να το εμφανίζετε μέσω των 7-Segment LED.
- Χρησιμοποιώντας τον Transmitter του UART, μπορείτε μέσω ενός τερματικού να στέλνετε συγκεκριμένα μηνύματα στον παίκτη, π.χ. όταν γίνεται game over να στέλνετε το μήνυμα "Game over!" και όταν ξεκινάει καινούριο παιχνίδι να στέλνετε το μήνυμα "New Game".
- Αντί να εμφανίζεται η αρχική κατάσταση του παιχνιδιού όταν γίνεται Game Over, μπορείτε, χρησιμοποιώντας μία Video RAM, να εμφανίζετε μία προφορτωμένη εικόνα, η οποία θα μπορεί να απεικονίζει, με κατάλληλο σχηματισμό των pixels, το μήνυμα "Game Over" με μεγάλα γράμματα.
- Μπορείτε να προσθέσετε επιλογή παιχνιδιού για δύο παίκτες (Multiplayer Mode), το οποίο απαιτεί την προσθήκη μίας δεύτερης ρακέτας στη δεξιά μεριά της οθόνης, και έξυπνο χειρισμό των τεσσάρων κουμπιών της πλακέτας προκειμένου να μπορεί να χειριστεί ο κάθε παίκτης τη δική του ρακέτα.
- Αντί η γωνία της ανάκλασης μετά από πρόσκρουση του δίσκου στη ρακέτα να είναι σταθερή, μπορεί να εξαρτάται από την κίνηση της ρακέτας τη στιγμή που την αγγίζει ο δίσκος. Μπορείτε να κάνετε χρήση τριγωνομετρικών εξισώσεων για να πετύχετε την επιλογή κατάλληλης γωνίας ανάκλασης, όπως γίνεται στο κανονικό παιχνίδι Pong.

(Θα δοθεί μόνον στην βαθμολογία για την υλοποίηση του προαιρετικού μέρους)

Όταν η κυκλωματική υλοποίηση είναι ορθή, επιδείξτε τον κώδικα Verilog που γράψατε για το κύκλωμα (RTL Design) και το πλαίσιο δοκιμής (Testbench), τα αποτελέσματα της προσομοίωσης, και προαιρετικά το κύκλωμα εν λειτουργία σε επιτηρητή του εργαστηρίου.

8 Προθεσμία Παράδοσης, Υποβολή της Εργασίας και Αναφορά

Η προθεσμία παράδοσης της 4^{ης} εργασίας είναι η **16/01/2026**.

Μέχρι την προθεσμία της εργασίας θα πρέπει:

- να έχετε επιδείξει όλα τα επιμέρους μέρη της εργασίας στους επιτηρητές,
- να έχετε υποβάλλει τον κώδικα σας (RTL Design και Testbench) για κάθε σκέλος υλοποίησης ξεχωριστά,
- να έχετε υποβάλλει μια εργαστηριακή αναφορά, βάση των σημειώσεων από το βιβλίο του εργαστηρίου, **με όλη την απαραίτητη πληροφορία** (*Dataflow, FSM State Graph(s), Simulation Screenshot(s), etc.*).

9 Ερωτήσεις και Απορίες

Για οποιεσδήποτε ερωτήσεις και απορίες εκμεταλλευτείτε τον ιστιότοπο του e-Class και το χρόνο του εργαστηρίου ρωτώντας τους επιτηρητές.